A1-10 a page shared by two processes

the short version – every line explained

Start: `task start a1-10`. **Done:** `task test` shows **9 of 9** and no leak. This code is tested (9 of 9 on your team's code). Type it yourself, one step at a time.

From the list of earlier years: *simple shm.* **Type S + E**: two syscalls + cleanup when a process ends.

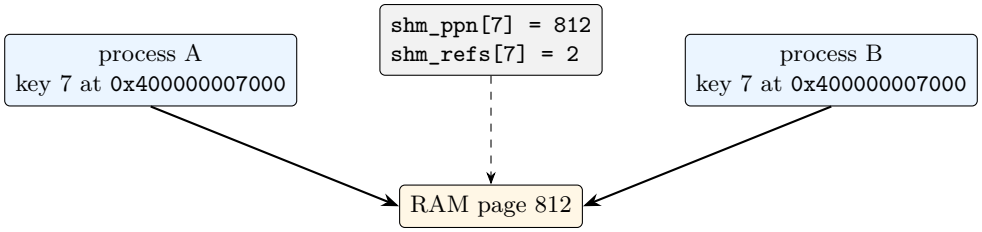
The task in 4 sentences

1. Normally every process has its own private memory – no other process can see it.
2. `shm_attach(7)` in two processes puts the **same piece of RAM** into both: what one writes, the other reads.
3. The kernel remembers, per key (1–15), **which piece of RAM** and **how many processes** use it.
4. The RAM is only given back when **nobody** uses it any more (count = 0).

The words you need

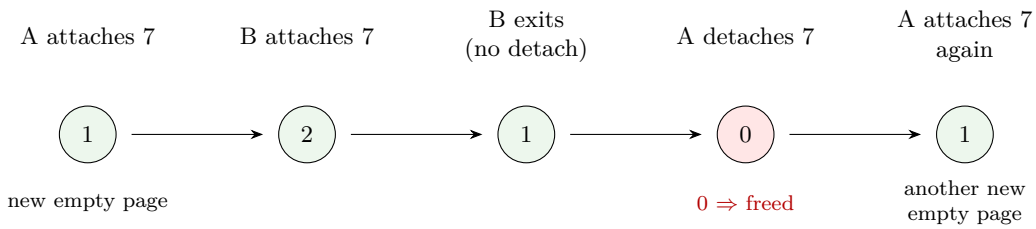
key	the number (1–15) two processes agree on: "I want shared page 7"
physical page (ppn)	a 4096-byte piece of real RAM, by its number
virtual page (vpn)	a 4096-byte piece of one process's addresses
map / unmap	enter / remove "vpn → ppn" in one process's page table
free	give the piece of RAM back to the kernel (`freePPN`)
count (refs)	how many processes have the key attached
leak	RAM that is never given back – the test checks for it at the end
lock	"only one thread at a time in here" – needed because many processes use the table

Picture 1 – one piece of RAM in two processes

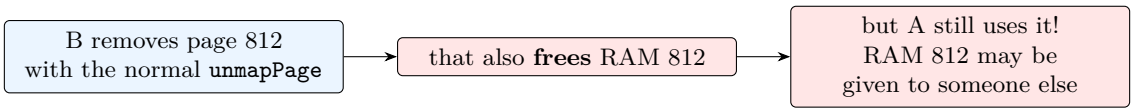


Both page tables point to the same RAM page. Two global arrays remember it: `shm_ppn` (which page) and `shm_refs` (how many users).

Picture 2 – the count over time (this is exactly the test)

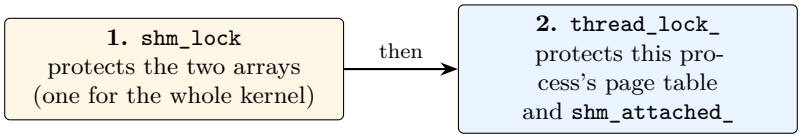


Picture 3 – the trap: unmap must not free



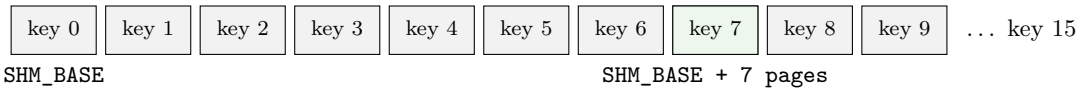
So: a new parameter for `unmapPage` – *remove, but do not free* (step 2). The same trap hides in `delete arch_memory_` at process exit: it frees *every* mapped page (step 7 handles it).

Picture 4 – two locks, always the same order



A deadlock needs two threads waiting for each other's lock in *opposite* order. Nothing else in the kernel takes `shm_lock`, and our code always takes it *first* – so that cannot happen.

Picture 5 – where the shared pages appear



Key k always appears at `SHM_BASE` + k pages, in every process. Key 0 is not used.

The 7 steps

#	What	File
1	plumbing: numbers, libc, `cases`	`syscall-definitions.h`, `nonstd.h/.c`, `Syscall.cpp`
2	unmap *without* freeing	`ArchMemory.h/.cpp`
3	constants + per-process state	`UserProcess.h`
4	the table: 2 arrays + 1 lock	`UserProcess.cpp` (top), `main.cpp`
5	attach	`UserProcess.cpp`
6	detach	`UserProcess.cpp`
7	cleanup when a process ends	`UserProcess.cpp` (destructor)

1 Plumbing

`common/include/kernel/syscall-definitions.h` – next to the other numbers:

```
1 #define sc_shm_attach 1680
2 #define sc_shm_detach 1681
```

1–2 The numbers of the two new syscalls. The program and the kernel both include this file, so they agree.

`userspace/libc/include/nonstd.h` – at the top `#include "types.h"` (for `size_t`); before the closing `#ifdef`:

```
1 extern void* shm_attach(size_t key);
2 extern int shm_detach(size_t key);
```

1 Declares `shm_attach`: it gives back an address (`void*`).

2 Declares `shm_detach`: it gives back 0 or −1. Without these two lines the test does not compile.

`userspace/libc/src/nonstd.c` – at the end:

```
1 void* shm_attach(size_t key)
2 {
3   return (void*) __syscall(sc_shm_attach, key, 0x00, 0x00, 0x00, 0x00);
4 }
5 int shm_detach(size_t key)
6 {
7   return __syscall(sc_shm_detach, key, 0x00, 0x00, 0x00, 0x00);
8 }
```

3 Enter the kernel with number 1680 and the key as first argument (the other four are unused). The result is a number – `(void*)` turns it into an address; 0 becomes `NULL`.

7 The same with 1681. The kernel's −1 arrives as −1 in the `int`.

`common/source/kernel/Syscall.cpp` – in the `switch` of `syscallException`, e.g. before `case sc_execv`:

```
1 case sc_shm_attach:
2   return_value = currentThread->getProcess()->shmAttach(arg1);
3   break;
4 case sc_shm_detach:
5   return_value = currentThread->getProcess()->shmDetach(arg1);
6   break;
```

1, 4 "If the program asked for 1680 / 1681 . . . "

2, 5 . . . call the method of the *calling* process with the key (`arg1`). `return_value` goes back to the program.

3, 6 End of the `case`.

Done when

The test compiles (it still fails – the methods come in step 5–6).

2 Unmap without freeing

`arch/x86/64/include/ArchMemory.h` – change the declaration of `unmapPage`:

```
1 bool unmapPage(uint64 virtual_page, bool free_physical_page = true);
```

1 A second parameter: free the RAM page or not. `= true` is the default, so every old call `unmapPage(vpn)` still frees, exactly as before. Only our code passes `false`.

`arch/x86/64/source/ArchMemory.cpp` – the first line of the function and the `freePPN` line:

```
1 bool ArchMemory::unmapPage(uint64 virtual_page, bool free_physical_page)
2 ...
3   if (free_physical_page)
4     PageManager::instance()->freePPN(m.page_ppn);
```

1 The same parameter, without the `= true` (the default belongs only in the `.h`).

3–4 Free the RAM page only if asked to. The rest of `unmapPage` (remove the entry) stays as it is.

Done when

`unmapPage(vpn, false)` removes the page from the page table but keeps the RAM (picture 3).

3 Constants and per-process state `common/include/kernel/UserProcess.h`

```
1 #define SHM_KEYS 16
2 #define SHM_BASE 0x0000400000000000ULL
3
4 size_t shmAttach(size_t key);
5 size_t shmDetach(size_t key);
6
7 bool shm_attached_[SHM_KEYS] = {};
```

1 *(at the top, next to the other `#defines`)* 16 slots: keys 1–15, slot 0 unused.

2 The address of "key 0"; key k is k pages above it (picture 5). Far away from code, heap and stacks.

4–5 *(in the `public:` part, e.g. after `cancelThread`)* The two methods the `case`s call. Public, so `Syscall.cpp` may call them.

7 *(in the `private:` part, at the bottom)* One flag per key: "*this* process has it mapped". `= {}` starts all as `false`. Needed to answer: attach twice? detach without attach? what to clean up at exit?

Done when

The constants, two declarations and one member are in `UserProcess.h`.

4 The table: two arrays and one lock

`common/source/kernel/UserProcess.cpp` – below the `#includes` (add `#include "ScopeLock.h"`):

```
size_t shm_ppn[SHM_KEYS] = {};
size_t shm_refs[SHM_KEYS] = {};
Mutex* shm_lock = nullptr;
```

1 Per key: which RAM page belongs to it. 0 = no page yet.

2 Per key: how many processes have it attached.

3 The lock for both arrays (picture 4). Only a *pointer*: SWEB never runs constructors of global objects, and a `Mutex` needs its constructor. Plain number arrays need none – they are just 0.

1–3 They are outside any class: one table for the whole kernel, shared by all processes.

`common/source/kernel/main.cpp` – `#include "Mutex.h"` at the top, then:

```
extern Mutex* shm_lock;

  shm_lock = new Mutex("shm_lock");
```

1 *(at file level, above `startup()`)* "This variable exists in another file" – so `main.cpp` may use it.

3 *(in `startup()`, right after `Scheduler::instance();`)* Create the lock once at boot, before any program can attach.

Done when

It compiles except for the two missing methods.

5 Attach

`UserProcess.cpp`, at the end

```
size_t UserProcess::shmAttach(size_t key)
{
  if (key == 0 || key >= SHM_KEYS)
    return 0;
  size_t vpn = SHM_BASE / PAGE_SIZE + key;
  ScopeLock global(*shm_lock);
  ScopeLock local(thread_lock_);
  if (!shm_attached_[key])
  {
    if (shm_refs[key] == 0)
      shm_ppn[key] = PageManager::instance()->allocPPN();
    shm_refs[key]++;
    bool mapped = arch_memory_->mapPage(vpn, shm_ppn[key], 1);
    assert(mapped);
    shm_attached_[key] = true;
  }
  return vpn * PAGE_SIZE;
}
```

3–4 Only keys 1–15 exist. Otherwise return 0 = `NULL` (the test tries 0 and 16).

5 The key's fixed virtual page number: `SHM_BASE` as a page number, plus the key (picture 5).

6 Take the table lock first …

7 … then this process's lock. Same order everywhere (picture 4). Both are released automatically at every `return`.

8 Only if this process does not have the key yet – attaching twice must not map twice or count twice.

10–11 Nobody uses the key yet: take a fresh piece of RAM. `allocPPN` fills it with zeros ("the first attach creates the page, zeroed").

12 One more user.

13 Enter "vpn → RAM page" in *this* process's page table; `1` = the program may access it. `mapPage` returns `false` only if the vpn is already mapped …

14 … which cannot happen (line 8 checked it) – the `assert` would catch a bug. (`mapPage` is `[[nodiscard]]`: its result must be used.)

15 Remember: this process has the key now.

17 The address = page number × 4096. Also for the second attach: the same address.

Done when

Attach maps the page once per process and counts each process once.

6 Detach

`UserProcess.cpp`

```
1  size_t UserProcess::shmDetach(size_t key)
2  {
3    if (key == 0 || key >= SHM_KEYS)
4      return (size_t) -1;
5    ScopeLock global(*shm_lock);
6    ScopeLock local(thread_lock_);
7    if (!shm_attached_[key])
8      return (size_t) -1;
9    arch_memory_->unmapPage(SHM_BASE / PAGE_SIZE + key, false);
10   ArchMemory::flushTlb();
11   shm_attached_[key] = false;
12   shm_refs[key]--;
13   if (shm_refs[key] == 0)
14     PageManager::instance()->freePPN(shm_ppn[key]);
15   return 0;
16 }
```

3–4 Invalid key → −1.

5–6 The two locks, same order as in attach.

7–8 Not attached in this process → −1 ("detaching twice fails").

9 Remove the page from this process's page table, **`false` = keep the RAM** – others may use it.

10 The CPU caches translations (TLB). Flush, so it forgets the old one and a later access really fails.

11 This process no longer has the key.

12 One user less.

13–14 Nobody left: now give the RAM back ("after the last detach the page was freed"). `shm_ppn[key]` keeps the old number, but `shm_refs` is 0 – the next attach takes a new page (line 10 of attach).

15 Success.

Done when

Detach removes the page and frees the RAM only for the last user.

7 Cleanup when a process ends

`UserProcess.cpp`, ~UserProcess

Between `delete loader_;` and `delete arch_memory_;`:

```
1  {
2    ScopeLock global(*shm_lock);
3    for (size_t key = 1; key < SHM_KEYS; ++key)
4    {
5      if (!shm_attached_[key])
6        continue;
7      arch_memory_->unmapPage(SHM_BASE / PAGE_SIZE + key, false);
8      shm_refs[key]--;
9      if (shm_refs[key] == 0)
10       PageManager::instance()->freePPN(shm_ppn[key]);
11   }
12 }
```

Why at all A program may end *without* `shm_detach` (the test's second child does). Then `delete arch_memory_` would free every mapped page – also the shared one (picture 3).

1, 12 An extra block: the lock of line 2 is released at line 12, before `delete arch_memory_`.

2 The table lock (we change `shm_refs`). No `thread_lock_`: the process is being deleted, none of its threads runs any more.

3 Go through all keys 1–15.

5–6 Skip the keys this process does not have.

7 Remove the page, keep the RAM – *before* `delete arch_memory_` could free it.

8–10 One user less; free the RAM if it was the last (otherwise: a leak at the end of the test).

Done when

`task test`: **9 of 9 passed**, `sweb_run status 0` (no leak).

The test – what each check means

Check	Made to pass by
attached key 7	steps 1–5
attaching twice gives the same address	attach, line 8 + 17
child sees the parent's text / wrote into our page	one RAM page in two page tables (picture 1)
page survives a child that did not detach	step 7 + `false` in step 2: count 2 → 1, RAM kept
detached	step 6
detaching twice fails	detach, lines 7–8
after the last detach the page was freed: a new one is zero	detach lines 13–14, then attach lines 10–11
invalid keys rejected	attach, lines 3–4
no leak at the end	step 7: the parent ends attached, its count is dropped and the RAM freed

Remember – and the tutor questions

Three sentences

1. One RAM page in several page tables; two arrays remember the page and **count the users** per key.
2. Removing a shared page **never frees** it – only the last user (count 0) frees it, also at process exit.
3. Two locks, **always `shm_lock` first**, then `thread_lock_` – that rules out a deadlock.

Question	Short answer
How do two processes see the same memory?	both page tables point to the same RAM page
Why count the users?	the RAM may only be freed when the last one is gone
Why not the normal `unmapPage`?	it frees the RAM, the other process would lose its page
A process exits without detaching?	the destructor unmaps without freeing and lowers the count
Why is the lock created in `startup()`?	SWEB does not run constructors of global objects
Why can't the two locks deadlock?	same order everywhere, and nothing else takes `shm_lock`
Why `flushTlb` in detach?	the CPU might still use the old translation
What would `fork` do with an attached page?	your team's fork copies the address space: the child gets a private copy, not the shared page